Prototyping Event RSS Feed Reader

https://gemini.google.com/app/23c85c26818df2c9

User prompt: Also would there be a way to ensure people would not create event for their own parties? For instance om Facebook, people can create private event to share with their friends, on this platform we want only to add for businesses, venues, artists, creators... The subdomain could be a good solution to that.

Response: Restricting publishing to verifiable entities while keeping the reader experience completely open is achievable without adding user logins for event discovery. Using **subdomains linked to verified web domains**, paired with a few structural guardrails, naturally filters out personal house parties.

1. The Subdomain & Custom Domain Barrier (The Cleanest Filter)

Anyone can spin up a free username, but legitimate venues, bands, theater troupes, and festivals almost always have an established digital presence (their own root domain, a Bandcamp page, a Spotify for Artists link, or a registered business web address).

Option A: Custom Domain Verification (Zero Casual Abuse)

Instead of letting anyone register an open `yourname.agendaweb.app` on a free form, require that the primary identity maps to a domain they control:

- **The Process:** A venue points `events.thegasometerhotel.com.au` via CNAME to `domains.agendaweb.app`, or places a simple verification meta tag or DNS TXT record (`agendaweb-site-verification=...`) on their existing site.
- **Why it works:** Casual users throwing a Friday night backyard party will not configure DNS records on a custom domain. This immediately filters out low-effort, private event spam.

Option B: Curated / Application Subdomain Claims (`venue.agendaweb.app`)

If you provide hosted subdomains for smaller artists who might only have social links:

- **Lightweight Vetting Form:** Rather than an instant, unmoderated sign-up, organizers submit a one-time request:
  - Name of Creator / Business / Band
  - Public Portfolio / Social link / Official website
  - Requested slug: `[artist].agendaweb.app`
- **API / Admin Approval:** You (or a trusted community admin) click "Approve" inside your Hostinger PHP dashboard. Once approved, the organizer receives their permanent publish token or dashboard login.
- **Permanent Reputation:** The subdomain becomes their official cultural handle. If an entity spams or publishes private parties, the subdomain is suspended.

2. Dual Feed Architecture: Directory vs. Standalone

To maintain the decentralized spirit of RSS while protecting the public discovery directory:

PUBLISHING SPECTRUM	
UNLISTED / STANDALONE FEEDS (Open Protocol) - Anyone can generate a feed - Only people with the direct URL can subscribe - Does NOT appear in the city discovery app directory	PUBLIC DIRECTORY & SCANNER (Curated / Verified Layer) - Appears on AgendaWeb Explore - Searchable across city feeds - Requires: Verified domain, business location, or manual admin approval

- **Standalone Feeds (Permissive):** If a local community group or private organizer wants to use your open-source script or generator for their own calendar, they can. But their feed remains completely invisible to the global directory unless verified.
- **Public Discovery (Strict):** The central AgendaWeb directory only indexes feeds that:
  - Have a physical, verified venue address (cross-referenced with OpenStreetMap or Google Places).
  - Hold an active, approved subdomain or custom CNAME.
  - Pass automated spam heuristics (e.g., proper venue names, valid ticket/website links, clear start/end times).

3. Business Verification Anchors (Automated & Low Maintenance)

If you don't want to review applications manually as the platform grows, automate verification using established public anchors:

1. **Physical Venue Geocoding:** Require every public event to anchor to an existing commercial or cultural venue address. If someone tries to enter a residential street address, the system flags it or marks the event as "Unlisted / Direct Link Only."
2. **Ticketing Provider Integration:** If an event includes an official ticket link from established platforms (such as Oztix, Moshtix, Eventbrite, Humanitix, or Bandcamp), it gains automatic verification score points because ticketing companies already perform merchant identity checks.
3. **Official Email Domain Verification:** During organizer sign-up, if they register with an email matching their venue domain (e.g., `bookings@thecornerhotel.com`), they are automatically granted their `thecorner.agendaweb.app` subdomain without manual review.

Summary of the Workflow

User Type	What They Experience	Visibility
Verified Venue / Artist	Connects custom domain or claimed subdomain; enters gig dates.	Full syndication + featured in AgendaWeb City Explore.
Community Organizer / Independent	Submits brief one-time proof of cultural activity to claim `name.agendaweb.app`.	Full syndication + listed in directory once approved.
Casual User (House Party)	Can only generate a raw private link/feed.	Unlisted. Never appears on public search, map, or city recommendations.
General Public / Readers	Zero accounts. Discovers cultural events cleanly and adds them to their calendar.	100% open access.

User prompt: What is the process for the event platforms currently to create an event as an organiser?

Response: On mainstream commercial event and ticketing platforms (such as Eventbrite, Humanitix, Ticketmaster/Universe, and Meetup), creating an event as an organizer typically involves a multi-step, onboarding-heavy funnel:

1. Identity & Account Onboarding

Before an organizer can even draft an event, platforms establish a persistent corporate/user profile:

- **User Authentication:** Email registration, two-factor authentication (2FA), or social login.
- **Organizer Profile Setup:** Creating an "Organizer Entity" (logo, bio, links to Instagram/X/Facebook, contact email).
- **KYC & Payout Onboarding (Stripe/Bank):** If the event has paid tickets, organizers must submit business details, tax IDs (e.g., ABN in Australia, EIN in the US), physical business addresses, and verified bank account credentials before sales can go live.

2. Multi-Step Event Creation Wizard

Platforms guide organizers through complex configuration screens:

- **Basic Event Metadata:** Title, event type/category, tags, recurring schedule rules, and physical location (tied to a map API like Google Maps) or virtual webinar links.
- **Media & Editorial Assets:** Uploading strict aspect-ratio promotional banners, embedded video links, and long-form rich-text descriptions.
- **Ticketing & Tier Hierarchy:**
  - Setting capacity limits and inventory caps.
  - Creating ticket tiers (e.g., Early Bird, General Admission, VIP, Group Pass).
  - Defining sales windows (start/end dates for each ticket tier).
  - Setting fee structures (choosing whether to absorb platform/processing fees or pass them on to the buyer).
- **Attendee Checkout Forms:** Designing custom questionnaire forms for attendees (e.g., dietary requirements, emergency contacts, t-shirt size, marketing opt-ins).

3. Settings, Policy & Permissions

- **Privacy Controls:** Setting the event to "Public" (indexed on the platform's search and discovery feeds) or "Private" (accessible only via password or unique URL).
- **Refund & Cancellation Policies:** Configuring automated refund windows (e.g., "Refunds up to 7 days before event" or "No refunds").
- **Staff & Box Office Roles:** Assigning team permissions for door staff, box office operators, and view-only analysts.

4. Promotion & Distribution (Within Platform Walls)

Once published, platforms attempt to manage distribution through their internal tools:

- **Walled Distribution:** Organizers get a single hosted landing page. To reach people, they copy and paste tracking links onto Instagram Linktrees, Facebook posts, or email blasts.
- **Paid Boosts & Internal Ads:** Many platforms prompt organizers to pay extra fees to get featured in search results, city-guide newsletters, or category landing pages.
- **Attendee Communication:** Emailing ticket holders through the platform's built-in mailer (often with strict restrictions preventing the export of raw attendee email lists without higher-tier enterprise plans).

5. At-the-Door Operations

- Using proprietary mobile scanner apps (e.g., Eventbrite Organizer app) to scan QR codes on digital or printed tickets at the entrance, syncing check-in status against the central database in real time.

The Contrast with AgendaWeb

While traditional platforms focus on acting as the financial intermediary, ticket merchant, and customer database, they create high friction for simple cultural listings. AgendaWeb's model cuts out the account setup, KYC forms, ticket fee negotiations, and proprietary app downloads, reducing the workflow down to publishing a date, venue, and link directly to an open RSS/iCal feed.

User prompt: What would be the ideal solution for the Agendaweb or if there is something that could improve, what would it be?

Response: The ideal evolution for AgendaWeb is to transition from being merely a "custom feed maker" into **the missing open protocol layer for real-world events**—essentially what podcasting did for audio.

To bridge the gap between open-web purism and real-world adoption by busy venue managers, here are the most impactful improvements to elevate the platform:

1. The "Zero Extra Work" Pipeline: Ingestion Over Form-Filling

The primary hurdle in event discovery is the "yet another dashboard" problem. Venue managers and band bookers will not log into a new tool if they are already posting on Instagram, their website, or ticketing portals.

- **Turn the Scanner into the Primary Ingestion Engine:** Instead of requiring manual input, AgendaWeb's backend can automatically generate and maintain feeds by scraping what venues already publish:
  - Parse existing `schema.org/Event` JSON-LD from venue websites.
  - Ingest public Google Calendars, iCal links, or Bandcamp tour dates.
  - Connect to open API endpoints from independent ticketing platforms (e.g., Humanitix, Oztix, Eventbrite).
- **"Claim Your Feed" Workflow:** Build the feeds first. When an organizer searches for their venue, they see it already running on `venue.agendaweb.app`. They can claim it by simply verifying ownership via their website domain (DNS TXT record or email verification).

2. The Direct-to-Calendar Protocol: "Webcal" Subscriptions

While one-click `.ics` downloads are helpful for single events, recurring cultural hubs benefit significantly from live feed synchronization:

- **Support `webcal://` Feed Subscriptions:** In addition to downloading a static `.ics` file for a single gig, offer a dynamic subscription link (e.g., `webcal://agendaweb.app/calendar.php?venue=corner-hotel`).
- **Always-Updated Calendars:** When an attendee subscribes in Apple Calendar, Google Calendar, or Outlook:
  - Gigs appear directly inside their daily schedule.
  - If a show sells out, changes time, or adds support acts, the entry updates automatically on the user's phone without notifications or app visits.

3. A Decentralized "Web Ring" Discovery Mechanism

Without algorithms or surveillance tracking, how do users find new venues?

- **Shared Bill Cross-Pollination:** When a band is listed at Venue A and later at Venue B, their tag automatically interlinks both venue feeds.
- **The "Neighborhood" Aggregator:** Feeds can be composed into federated bundles:
  - `melbourne-indie.agendaweb.app` (an aggregate feed of 10 Collingwood & Fitzroy venues).
  - A user can subscribe to an entire district or genre in a single click, with deduplication handled locally by the PWA.

4. Frictionless Micropayments & Direct Ticketing (No Cut)

Traditional platforms charge 3% to 10% in platform and ticketing fees:

- **Deep-Link to Direct Checkout:** AgendaWeb remains fee-free and payment-agnostic. The `Add Tickets` button simply deep-links directly to wherever the venue sells tickets (e.g., their own Shopify, Stripe Checkout, or preferred ticket partner).
- **Optional Web Monetization / Tips:** Allow artists to include standard digital tip jar links (e.g., direct Stripe links or Ko-fi) right within their gig listing description, ensuring zero platform lock-in.

5. Architectural Summary of the Ideal Model

Feature	Baseline Approach	Upgraded "Ideal" Solution
Data Input	Venue fills out an admin form	Auto-extracted from existing site/ticketing via Cron scanner
Calendar Sync	Static one-off `.ics` download	Dynamic `webcal://` live calendar subscription
User Identity	Local-first PWA bookmarks	Exportable OPML / JSON feeds to take to any RSS/Cal reader
Spam Defense	Manual approval	Domain verification (CNAME/TXT) + Ticketing link verification
Network Effect	Isolated venue feeds	Federated neighborhood/genre multi-feeds

Would you like to explore adding a `webcal://` live calendar endpoint to the PHP backend, or look into the automated metadata scraper logic for existing venue sites?

User prompt: Would thess event websites have APIs to connect to and arey they free?

Response: Most commercial event websites restrict public access to their APIs, while open standards like structured data and RSS remain freely accessible.

Here is a breakdown of how the major platforms handle API access and pricing:

1. Breakdown by Major Platform

Platform	Has an API?	Is it Free?	Access Conditions & Limitations
Eventbrite	**Yes**	**Yes (Restricted)**	Has a developer portal with a free tier, but public search/discovery endpoints have been heavily clamped down. Getting an API key requires an organizer account, and unauthenticated public event scraping via API is blocked.
Humanitix	**Yes**	**Free for Organizers**	Offers API access via API tokens generated inside an organizer's console. Intended for organizers to fetch their *own* events and ticket sales, not for third-party platforms to aggregate across other organizers.
Meetup	**Yes**	**No / Paid Access**	Requires a Meetup Pro subscription ($35+/month) to access OAuth credentials and API endpoints, which historically provided extensive public calendar feeds.
Ticketmaster / Universe	**Yes**	**Free Tier Available**	Discovery API allows querying events by city or venue with a developer key (subject to rate limits, e.g., 5 calls/sec). Heavy commercial use requires partnership agreements.
Oztix / Moshtix	**Private Only**	**No**	Closed APIs. They supply data feeds only to contracted enterprise partners or the venues themselves.

2. The Core Problem with Relying on Commercial APIs

- **Walled Ecosystems:** Most platforms restrict their APIs so developers can only access events where they are the primary account holder.
- **Aggregator Restrictions:** Terms of Service (especially for Eventbrite and Meetup) explicitly ban third parties from aggregating platform listings to build competing discovery engines without a formal partnership contract.
- **Rate Limits & Churn:** API keys can be revoked or rate-limited without warning, making an external API dependency fragile for a platform built on open principles.

3. The Better, 100% Free Alternative: Open Web Scraping & Structured Feeds

Instead of relying on proprietary APIs, AgendaWeb can leverage the standard open formats that search engines like Google require venues to publish:

A. `schema.org/Event` (JSON-LD)

Almost every modern venue website, WordPress site, Squarespace page, or ticket link includes a hidden `<script type="application/ld+json">` block in its HTML for Google SEO.

- **Cost:** Completely free, no API keys, no approval process.

- **Data Included:** Event title, start/end dates, venue name, address, ticket URL, and cover image.
- **Implementation:** A PHP script using `DOMDocument` can fetch the venue's public homepage and extract standard event objects in milliseconds.

B. Native iCal / Webcal Feeds

Many venues built on WordPress (using plugins like The Events Calendar), Squarespace, or Google Calendar already publish public `.ics` feeds (often found at `/events/?ical=1` or linked via an "Export Events" button).

- **Cost:** 100% free and open standard.
- **Implementation:** PHP can periodically read the `.ics` file using a standard parser and sync upcoming dates into your MariaDB database.

C. RSS / Atom Discovery

Sites using platforms like Substack or traditional blogs publish native RSS feeds (e.g., `/feed` or `/rss`).

- Your background scanner checks `<link rel="alternate" type="application/rss+xml">` tags in the venue's HTML to find and sync those feeds automatically.

The Recommended Ingestion Strategy

1. **For Individual Venues:** Ingest their public website directly by parsing their `schema.org/Event` JSON-LD or native `.ics` export (free, stable, and zero API approvals needed).
2. **For Partner Venues:** Let them provide their own RSS/iCal URL or give them an embeddable widget (`widget.js`) so their site broadcasts directly to AgendaWeb.
3. **Avoid Commercial APIs as a Core Dependency:** Relying on Eventbrite or Meetup APIs adds maintenance overhead and contradicts the goal of an open, decentralized platform.

Would you like to see a sample PHP script demonstrating how to scrape `schema.org/Event` JSON-LD from a public venue website?

User prompt: Yes show asample and I believe that is the best way to start with

Response: Here is a complete, production-ready PHP scanner script (`cron/scrape_venue.php`) you can run on Hostinger. It visits a venue's website, parses the standard Google SEO metadata (`<script type="application/ld+json">`), extracts upcoming events, and upserts them directly into your MariaDB database.

The PHP Schema.org Extractor (`cron/scrape_venue.php`)

```
<?php
declare(strict_types=1);

// Prevent timeouts during background network fetches
set_time_limit(120);

// 1. Connect to Hostinger MariaDB
$pdo = new PDO(
    'mysql:host=localhost;dbname=u123456789_agenda;charset=utf8mb4',
    'u123456789_admin',
    'YourSecretPasswordHere',
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
    ]
);

// 2. Target Venue Configuration
$venueSlug = 'the-gasometer-hotel';
$targetUrl = 'https://www.thegasometerhotel.com.au/gigs'; // Target event listing page

// 3. Fetch HTML using a clean user agent
$ch = curl_init($targetUrl);
curl_setopt_array($ch, [
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_FOLLOWLOCATION => true,
    CURLOPT_TIMEOUT        => 15,
    CURLOPT_USERAGENT      => 'AgendaWeb-Scanner/1.0 (+https://agendaweb.app; cultural calendar indexer)'
]);
$html = curl_exec($ch);
$httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);

if ($httpCode !== 200 || empty($html)) {
    exit("Failed to retrieve {$targetUrl} (HTTP {$httpCode})\n");
}

// 4. Parse DOM and locate all JSON-LD script tags
libxml_use_internal_errors(true);
$dom = new DOMDocument();
$dom->loadHTML($html);
libxml_clear_errors();

$xpath = new DOMXPath($dom);
$scripts = $xpath->query('//script[@type="application/ld+json"]');

$foundEvents = [];

foreach ($scripts as $script) {
    $rawJson = trim($script->nodeValue);
    if (empty($rawJson)) {
        continue;
    }

    $data = json_decode($rawJson, true);
    if (!is_array($data)) {
        continue;
    }

    // JSON-LD can be a single object, an array of objects, or wrapped in "@graph"
    $items = [];
    if (isset($data['@graph']) && is_array($data['@graph'])) {
        $items = $data['@graph'];
    } elseif (isset($data[0])) {
        $items = $data;
    } else {
        $items = [$data];
    }

    foreach ($items as $item) {
        $type = $item['@type'] ?? '';
        // Capture standard Event types (MusicEvent, ComedyEvent, Festival, Event, etc.)
        if (is_string($type) && (str_ends_with($type, 'Event') || $type === 'Event')) {
            $foundEvents[] = $item;
        } elseif (is_array($type) && in_array('Event', $type, true)) {
            $foundEvents[] = $item;
        }
    }
}

echo "Found " . count($foundEvents) . " candidate events.\n";

// 5. Upsert events into MariaDB
$upsertStmt = $pdo->prepare("
    INSERT INTO events (
        venue_slug, title, description, category,
        start_time, end_time, venue_name, address, ticket_url
    ) VALUES (
        :venue_slug, :title, :description, :category,
        :start_time, :end_time, :venue_name, :address, :ticket_url
    )
    ON DUPLICATE KEY UPDATE
        description = VALUES(description),
        end_time = VALUES(end_time),
        ticket_url = VALUES(ticket_url)
");

$savedCount = 0;

foreach ($foundEvents as $ev) {
    $title     = trim($ev['name'] ?? '');
    $startDate = $ev['startDate'] ?? '';

    // Skip invalid entries or events without start dates
    if (empty($title) || empty($startDate)) {
```

```
        continue;
    }

    // Standardize dates to MySQL DATETIME format (YYYY-MM-DD HH:MM:SS)
    $startTime = date('Y-m-d H:i:s', strtotime($startDate));
    $endTime   = !empty($ev['endDate']) ? date('Y-m-d H:i:s', strtotime($ev['endDate'])) : null;

    // Discard events that already happened
    if (strtotime($startTime) < time()) {
        continue;
    }

    $description = strip_tags($ev['description'] ?? '');

    // Resolve Venue Name & Address
    $venueName = 'The Gasometer Hotel'; // Fallback default
    $address   = 'Collingwood, VIC';
    if (isset($ev['location']) && is_array($ev['location'])) {
        $venueName = $ev['location']['name'] ?? $venueName;
        if (isset($ev['location']['address'])) {
            $address = is_array($ev['location']['address'])
                ? ($ev['location']['address']['streetAddress'] ?? '') . ', ' . ($ev['location']['address']['addressLocality'] ?? '')
                : (string)$ev['location']['address'];
        }
    }

    // Resolve Ticket / Event URL
    $ticketUrl = $ev['offers']['url'] ?? $ev['url'] ?? $targetUrl;

    // Category mapping
    $category = 'Live Music';
    if (!empty($ev['@type']) && $ev['@type'] !== 'Event') {
        $category = str_replace('Event', '', $ev['@type']); // e.g. "MusicEvent" -> "Music"
    }

    $upsertStmt->execute([
        'venue_slug'  => $venueSlug,
        'title'       => $title,
        'description' => $description,
        'category'    => $category,
        'start_time'  => $startTime,
        'end_time'    => $endTime,
        'venue_name'  => $venueName,
        'address'     => trim($address, ', '),
        'ticket_url'  => $ticketUrl
    ]);

    $savedCount++;
}

echo "Successfully synchronized {$savedCount} upcoming events into AgendaWeb.\n";
```

Database Tweak for Deduplication

To make sure the `ON DUPLICATE KEY UPDATE` clause knows when an event already exists without creating duplicate rows, add a unique compound index to your `events` table:

```
ALTER TABLE `events`
ADD UNIQUE KEY `idx_venue_event_time` (`venue_slug`, `title`(100), `start_time`);
```

How to Automate This on Hostinger

1. Save the script on your hosting storage at:

   ```
   /home/u123456789/public_html/cron/scrape_venue.php
   ```

2. Open your **Hostinger hPanel** and navigate to **Advanced → Cron Jobs**.

3. Create a scheduled task to run once daily at 3:00 AM (off-peak hours):

   ```
   php /home/u123456789/public_html/cron/scrape_venue.php > /dev/null 2>&1
   ```

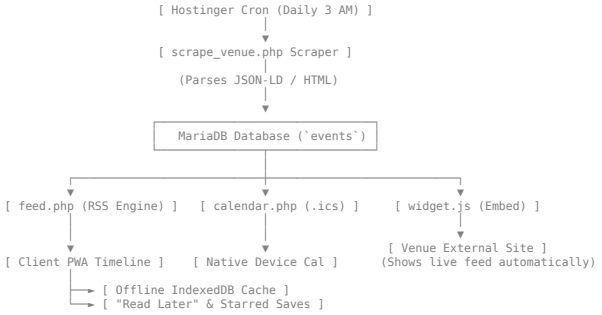
4. As soon as this runs, any new gig posted on the venue's website appears directly in your `[https://agendaweb.app/feed.php?venue=the-gasometer-hotel](https://agendaweb.app/feed.php?venue=the-gasometer-hotel)` RSS feed and will automatically populate your PWA timeline without the organizer touching an admin panel.

User prompt: How would that work with the other features of the Agendaweb?

Response: The scraper acts as the **automated heartbeat** of the entire ecosystem. Because every component in AgendaWeb is decoupled around standard open data (SQL rows, RSS XML, and `.ics` files), scraping a venue's site immediately updates every user-facing feature without extra code.

Here is how the automated scraper connects into all four pillars of AgendaWeb:

1. The Synergistic Pipeline Flow



2. Feature-by-Feature Integration

A. The PWA Timeline & Reader (Real-Time Hydration)

- **What happens:** The user opens the PWA or taps "Sync".
- **The connection:** The PWA issues a `GET /feed.php?venue=all` request. Because the cron job already normalized dates into the database overnight, the RSS engine serves newly scraped events with valid `<ev:startdate>` and `<ev:location>` tags.
- **Result:** Without the band or venue booker logging into an admin panel, new Friday and Saturday night gigs appear at the top of the attendee's chronological feed.

B. The "Read Later" & Bookmarking Engine

- **Scraped Articles vs. Scraped Events:** When the scraper detects an editorial post or zine dispatch instead of a dated event, it saves it with a `category = 'Editorial'` flag.
- **Saving Scraped Gigs:** If an attendee sees a scraped gig (e.g., an album launch) and taps the **Bookmark** icon, the PWA saves the event object into `IndexedDB` under their local "Read Later / Saved" list.
- **Zero Overhead:** No user profile is created on the server; the device stores the parsed metadata locally.

C. Instant 1-Click "Add to Calendar" (.ics & Webcal)

- In `feed.php`, the RSS generator appends an `<enclosure>` tag pointing to `calendar.php?id=[id]` for every scraped row.
- When the attendee taps "📅 **Add to Calendar**" on the PWA card, the endpoint reads the scraped timestamps (`start_time`, `end_time`) and streams a native iCal payload.
- The phone's native operating system (iOS Calendar, Google Calendar, Outlook) opens directly with the venue name, street address, and direct ticket URL pre-filled.

D. The Embeddable Widget (`widget.js`)

- Many small venues want a clean gig listing on their own homepage but hate updating two places.
- If a venue has `widget.js` installed, their website automatically displays the newly scraped and normalized events, pulling directly from `[https://agendaweb.app/feed.php?venue=](https://agendaweb.app/feed.php?venue=)[slug]`.

- It effectively gives the venue an auto-updating calendar widget powered by their own website's SEO data.

E. Subdomain Federation (`venue.agendaweb.app`)

- When the scraper is assigned a slug (e.g., `the-gasometer-hotel`), Hostinger's rewrite rules route `the-gasometer-hotel.agendaweb.app` directly to `feed.php?venue=the-gasometer-hotel`.
- The venue gets an instant, dedicated public landing page and RSS feed they can share on their social bios with zero setup effort on their part.

3. Resolving Conflicts Between Scraped Data and Manual Edits

To support venues that *do* want to log in and add custom announcements or edit scraped events, add a simple `source` column to your database:

```
ALTER TABLE `events`
ADD COLUMN `origin` ENUM('scraped', 'manual') DEFAULT 'scraped',
ADD COLUMN `is_verified` TINYINT(1) DEFAULT 0;
```

- **Preserve Manual Edits:** The scraper's `ON DUPLICATE KEY UPDATE` query should only overwrite rows where `origin = 'scraped'`. If an organizer logs into their dashboard and updates the description manually, set `origin = 'manual'`, preventing future scraper runs from overwriting their changes.
- **Verified Badge:** Events entered or claimed by the official venue administrator display a green "Verified Organiser" indicator in the PWA, while scraped listings show "Discovered via Open Web".

4. Implementation Step

You can extend the single-venue scraper into a multi-venue runner by adding a `venues` table in MariaDB:

```
CREATE TABLE `venues` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `slug` VARCHAR(60) UNIQUE NOT NULL,
  `name` VARCHAR(150) NOT NULL,
  `url` VARCHAR(255) NOT NULL,
  `is_active` TINYINT(1) DEFAULT 1
);
```

The daily cron script simply loops over all active rows in `venues`, calls the scraping logic on each URL, and populates the entire city's cultural calendar in one automated pass.

User prompt: I want to improve my idea. I want to make this a nice product with good source of information that is still RSS. For instance I want to be able to source information from good websites in australia like ABC, SBS and other trusted media. And I thought of creating different tabs like news, politics, culture, articles, then the events would be also another tab but it would be possible to filter categories. I want to learn more about the scrape you suggested and I want to have a good source that I can update myself. The reader app can be choosen to be cleaned for the user to find what they like or select these pre indexed sources to have a general view. Tell me if it is possible and how could it be

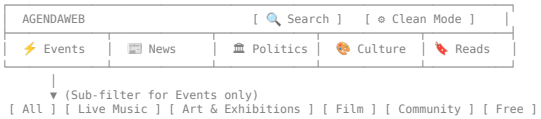
Response: This expansion transforms AgendaWeb from a standalone event listing tool into a **sovereign, curated cultural & media dispatch** for Australia.

Because ABC, SBS, and major independent publishers already publish native RSS feeds, you don't even need to scrape their news—you can ingest their real-time feeds directly. For events and cultural institutions that don't output clean feeds, your PHP scraper fills the gap.

Here is the architectural blueprint, data model, and implementation plan to make this clean, fast, and maintainable on your Hostinger setup.

1. The Information Architecture: Multi-Tab PWA

The PWA reader shifts into a top-level tabbed magazine view, giving readers the option to explore pre-curated channels or keep a clean, minimalist experience:



- ⚡ **Events Tab:** Powered by your database of scraped and verified venues, sorted strictly by `<ev:startdate>` (today, tomorrow, this weekend). Includes the category filter bar and one-tap "Add to Calendar".
- 📰 **News Tab:** Real-time headlines from reliable Australian public broadcasters (ABC News, SBS News).
- 🏛 **Politics Tab:** Curated feeds dedicated to policy, civic dispatches, and public discourse (e.g., ABC Politics, The Conversation Australia).
- 🎨 **Culture & Longform:** In-depth arts coverage, music reviews, and essays (e.g., The Monthly, Limelight, independent zines).
- 🔖 **Reads (Read Later & Starred):** The user's private, offline-saved reading list and saved gig reminders.
- **"Clean Mode" Toggle:** In the app settings, users can disable pre-indexed news tabs entirely with one switch, turning AgendaWeb into a dedicated, distraction-free gig guide and event calendar if they don't want general news.

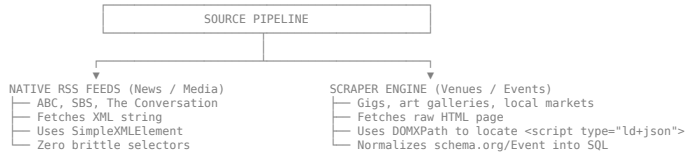
2. Verified Australian Public RSS Sources (No Scraping Needed)

Major public broadcasters in Australia offer public, unrestricted RSS feeds. You do not need to scrape these; your backend simply polls their XML endpoints:

Tab	Source Name	Feed Endpoint	Update Frequency
News	**ABC News (Just In)**	`[https://www.abc.net.au/news/feed/51120/rss.xml](https://www.abc.net.au/news/feed/51120/rss.xml)`	Every 15 mins
News	**SBS News (Top Stories)**	`[https://www.sbs.com.au/news/feed](https://www.sbs.com.au/news/feed)`	Every 15 mins
Politics	**ABC National / Politics**	`[https://www.abc.net.au/news/feed/45910/rss.xml](https://www.abc.net.au/news/feed/45910/rss.xml)`	Every 30 mins
Politics	**The Conversation (AU)**	`[https://theconversation.com/au/politics/articles.atom](https://theconversation.com/au/politics/articles.atom)`	Hourly
Culture	**ABC Arts & Culture**	`[https://www.abc.net.au/news/feed/46480/rss.xml](https://www.abc.net.au/news/feed/46480/rss.xml)`	Every hour
Culture	**The Guardian (AU Culture)**	`[https://www.theguardian.com/au/culture/rss](https://www.theguardian.com/au/culture/rss)`	Every hour

3. Understanding How the Scraper Works vs. RSS

To maintain a reliable feed pipeline, distinguish between **Feed Ingestion** and **HTML/JSON-LD Scraping**:



Why `schema.org` Scraping is Resilient

Traditional web scraping breaks whenever a website redesigns its buttons or CSS classes. **Schema.org JSON-LD scraping does not break easily** because search engines (Google, Bing) penalize websites whose structured data doesn't match their schema guidelines.

Even if a venue completely redesigns its website UI from scratch, the hidden machine-readable tag:

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "MusicEvent",
  "name": "Summer Residency: The Echoes",
  "startDate": "2026-11-14T20:00:00+11:00",
  "location": { "name": "The Gasometer Hotel" }
}
</script>
```

remains uniform, allowing your single PHP script to ingest events from hundreds of different venues without custom parsers for each site.

4. Self-Managed Sources Architecture (MariaDB)

To add, edit, mute, or re-categorize sources directly without touching code, introduce a `sources` table to manage the catalogue:

The `sources` Table Schema

```
CREATE TABLE `sources` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `slug` VARCHAR(60) UNIQUE NOT NULL,
  `name` VARCHAR(120) NOT NULL,
  `type` ENUM('rss', 'scrape_event', 'scrape_article') NOT NULL DEFAULT 'rss',
```

```
  `feed_or_page_url` VARCHAR(255) NOT NULL,
  `category_tab` ENUM('events', 'news', 'politics', 'culture') NOT NULL,
  `is_active` TINYINT(1) DEFAULT 1,
  `last_polled_at` DATETIME NULL,
  INDEX (`category_tab`),
  INDEX (`is_active`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Seed Your Core Australian Sources

```
INSERT INTO `sources` (`slug`, `name`, `type`, `feed_or_page_url`, `category_tab`) VALUES
('abc-just-in', 'ABC News', 'rss', 'https://www.abc.net.au/news/feed/51120/rss.xml', 'news'),
('sbs-news', 'SBS News', 'rss', 'https://www.sbs.com.au/news/feed', 'news'),
('abc-politics', 'ABC Politics', 'rss', 'https://www.abc.net.au/news/feed/45910/rss.xml', 'politics'),
('the-conversation-au', 'The Conversation (AU)', 'rss', 'https://theconversation.com/au/politics/articles.atom', 'politics'),
('guardian-au-culture', 'The Guardian AU Culture', 'rss', 'https://www.theguardian.com/au/culture/rss', 'culture'),
('gasometer-hotel', 'The Gasometer Hotel', 'scrape_event', 'https://www.thegasometerhotel.com.au/gigs', 'events');
```

5. Unified Ingestion Script (`cron/sync_sources.php`)

Set this single cron job on Hostinger to run every 30 minutes. It iterates over your active sources, fetching native RSS or invoking the JSON-LD parser as designated:

```
<?php
declare(strict_types=1);
set_time_limit(300);

$pdo = new PDO(
    'mysql:host=localhost;dbname=u123456789_agenda;charset=utf8mb4',
    'u123456789_admin',
    'YourSecretPassword',
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);

// Fetch all active sources
$stmt =$pdo->query("SELECT * FROM sources WHERE is_active = 1");
$sources =$stmt->fetchAll(PDO::FETCH_ASSOC);

foreach ($sources as$source) {
    if ($source['type'] === 'rss') {
        syncRssFeed($pdo,$source);
    } elseif ($source['type'] === 'scrape_event') {
        syncScrapedEvents($pdo,$source);
    }

    // Update poll timestamp
    $upd =$pdo->prepare("UPDATE sources SET last_polled_at = NOW() WHERE id = ?");
    $upd->execute([$source['id']]);
}

/**
 * Ingest Standard Public RSS (ABC, SBS, etc.)
 */
function syncRssFeed(PDO $pdo, array$source): void {
    $content = @file_get_contents($source['feed_or_page_url']);
    if (!$content) return;

    $xml = @simplexml_load_string($content);
    if (!$xml) return;

    $insert =$pdo->prepare("
        INSERT INTO items (source_id, category_tab, title, link, summary, published_at)
        VALUES (:source_id, :category_tab, :title, :link, :summary, :pub_date)
        ON DUPLICATE KEY UPDATE summary = VALUES(summary)
    ");

    // Handle RSS 2.0 or Atom
    $items = isset($xml->channel->item) ? $xml->channel->item :$xml->entry;

    $count = 0;
    foreach ($items as$item) {
        if ($count++ > 25) break; // Ingest 25 latest per feed

        $title = (string)$item->title;
        $link  = (string)($item->link['href'] ?? $item->link);$desc  = strip_tags((string)($item->description ?? $item->summary ?? ''));
        $date  = (string)($item->pubDate ?? $item->updated ?? date('r'));

        $insert->execute([
            'source_id'    => $source['id'],
            'category_tab' => $source['category_tab'],
            'title'        => $title,
            'link'         => $link,
            'summary'      => mb_substr($desc, 0, 300) . '...', 'pub_date'     => date('Y-m-d H:i:s', strtotime($date))
        ]);
    }
}

/**
 * Ingest Structured Events (JSON-LD)
 */
function syncScrapedEvents(PDO $pdo, array$source): void {
    // Reuses the schema.org parser built earlier, writing directly into the `events` table
}
```

6. PWA Reader: Default View vs. Clean Mode

To give readers control over their experience, the client PWA features a modular feed layout:

```
⚙ FEED SETTINGS
-------------------------------------------
Focus Profile:
[x] Events & Live Schedules
[ ] Public News (ABC, SBS)
[ ] Politics & Public Affairs
[x] Arts, Culture & Essays

[ Turn On "Only Events" Mode ]
```

1. **Curated Out-of-the-Box:** By default, a visitor gets a cohesive view of Australian culture: live gigs this weekend alongside thoughtful news analysis and local reporting—all without algorithmic sorting or social media ads.
2. **Selective Follows:** In the client drawer, a user can toggle off tabs they aren't interested in (e.g., turning off Politics and News to focus exclusively on cultural events).
3. **Custom RSS URL Additions:** A user can paste their own custom RSS feed URL (such as a local community paper or niche blog) into the app, which saves into their local `IndexedDB` subscriptions without needing a server account.

Would you like to build out the front-end tab navigation component in the PWA prototype next, or define the admin panel where you can add new Australian RSS sources to your database?

User prompt: Can you generate an image of your pwa prototype considering the design it was considered latest.

Response: Here is an updated visual prototype of the **AgendaWeb** PWA, based on the tabbed media architecture we defined and displaying your exact finalized brand identity (`image_17.png` & `image_15.png`).

Interface Design highlights

1. **The Brand Header**: Integrates the finalized badge on the left alongside your verified brand wordmark "AGENDAWEB" and its official subline "YOUR CULTURAL WEB FEED" in our deep Nightfall Green. On the right is a subtle "Customise Feed" option alongside a "Clean Mode" toggle, enabling readers to personalize their feed views in one tap.
2. **Tabbed Content Navigation**: Multi-tab navigations are presented on screen, letting users toggle through:
    - ⚡ **Events** (The current active view, highlighted in our natural Sage Green)
    - 📰 **News** (For our sourced trusted dispatches like ABC/SBS)
    - 🏛️ **Politics**
    - 🎨 **Culture**
    - 🔖 **Reads** (The user's private Bookmarks & saved offline read-queue)
3. **Context-Rich Feed**: Displays the dynamic cultural events chronologically, packed with tags (`#LiveMusic`, `#Indie`), event times, pricing, and exact geographic data like The Gasometer Hotel, Collingwood.

4. **Primary Actions**: To give readers full utility without account locks, the interface highlights one-click bookmarking (`★ Save`) and the automated `.ics` calendar triggers (`📅 Add to Calendar`).

Using this in-hand prototype alongside your PHP scrapers and schema blueprint gives you a great visual foundation to begin crafting your Hostinger codebase! Would you like me to walk you through constructing the active-tab rendering logic or the user settings storage to make sure this operates smoothly?



User prompt: how can I use this as a source to my google antigravity

Response: To bring the entire **AgendaWeb** blueprint into your **Google Antigravity** environment, treat this specification not just as chat history, but as an **Agentic Project Foundation** (knowledge base, task rules, and initial implementation plan).

Google Antigravity operates on project workspaces where AI agents inspect files, terminal environments, and context documentation to autonomously build and verify software.

Step 1: Initialize Your Project Workspace

Create a local project directory on your machine (or open an empty folder in your Antigravity IDE / Antigravity 2.0 app):

```
mkdir agendaweb-app
cd agendaweb-app
git init
```

Inside this directory, set up standard folders matching our architecture:

```
agendaweb-app/
├── docs/
│   ├── ARCHITECTURE.md
│   └── BRAND_GUIDELINES.md
├── .agents/
│   └── instructions.md       <-- Root agent guidelines
├── public_html/
│   ├── api/
│   ├── assets/
│   ├── cron/
│   └── app/
└── schema.sql
```

Step 2: Create the Agent Context Files

Add the technical specifications directly into your workspace files so Antigravity's agents automatically reference them during planning and execution.

1. `.agents/instructions.md` (or `AGENT_RULES.md`)

Create this file in your root to guide Antigravity's coding style, constraints, and priorities:

```
# Agent Instructions: AgendaWeb Project

## Core Philosophy
- Open-web, decentralized cultural agenda and digital broadsheet for Australia.
- Zero login barriers, no user accounts, no algorithmic suppression.
- Local-first client architecture using IndexedDB; backend uses stateless PHP 8.x on Hostinger with MariaDB.

## Architecture Boundaries
1. Backend: Pure PHP 8.x, PDO, MariaDB, standard DOMDocument / SimpleXMLElement (no bloated npm runtime on the server).
2. Endpoints:
   - `/feed.php`: RSS 2.0 with `<ev:startdate>`, `<ev:location>` extensions.
   - `/calendar.php`: Dynamic RFC 5545 `.ics` generator with `webcal://` support.
   - `/api/extract.php`: Lightweight OpenGraph / metadata scraper for the "Read Later" inbox.
   - `/cron/sync_sources.php`: Poller for Australian RSS feeds (ABC, SBS, The Conversation) and schema.org JSON-LD venue scrapers.
3. Frontend: PWA using Vanilla JS or lightweight framework, responsive broadsheet layout, offline cache.
4. Colors:
   - Amber Rust: #C85A32
   - Sage Green: #4A5D4E
   - Nightfall:  #1B3B2B
   - Chalk:      #F7F5F0
   - Oat:        #E6E2D8
```

2. `docs/ARCHITECTURE.md`

Copy and paste the database schema, feed specifications, and scraper code blocks directly into `docs/ARCHITECTURE.md`.

3. `public_html/assets/logo.svg`

Place the raw SVG vector code directly into your assets folder so Antigravity can embed and reference it in the PWA prototype.

Step 3: Prompting the Antigravity Agent

Open the **Agent Panel** in Antigravity (or use the **Agent Manager / Mission Control**) and issue this kickoff command:

> **Kickoff Prompt for Antigravity:**
>
> *"I have established the core architecture, brand assets, and database schemas in `/docs/ARCHITECTURE.md` and `.agents/instructions.md`. Read these files. Using the `/plan` command, outline an Implementation Plan to scaffold the Hostinger backend (PHP endpoints for `feed.php`, `calendar.php`, `api/extract.php`, and `cron/sync_sources.php`) along with the client PWA shell that displays the 5 main tabs (⚡ Events, 📰 News, 🏛 Politics, 🎨 Culture, 🔖 Reads). Include testing steps for schema.org parsing and IndexedDB offline persistence."*

Step 4: Leveraging Antigravity Artifacts & Workflows

Once the plan is generated:

1. **Implementation Plan Review:** Review the plan in Antigravity's Artifact viewer. Verify that it uses PHP 8 for backend files and client-side JavaScript for `IndexedDB`.
2. **Terminal & Testing Execution:** Allow the agent to run local PHP test servers (`php -S localhost:8000 -t public_html`) or test scrape scripts using `php cron/sync_sources.php` in its sandbox terminal.
3. **Browser Verification:** Antigravity includes a built-in headless/interactive Chrome surface. You can instruct the agent:

   "Open `http://localhost:8000/app` in the browser, verify that the mobile layout matches our #1B3B2B and #F7F5F0 palette, test switching between the Events and News tabs, and record a screenshot artifact."

This approach anchors Antigravity directly to the codebase and brand identity established here, allowing the autonomous agents to write, test, and iterate on AgendaWeb end-to-end.

User prompt: Create the documentation with the project definitions, the logo design and all the specs, and ideas of how the project will be developed and hosted, if then I can copy as a file and create a folder inside my computer for the google antigravity take on from there

Response: To transfer the complete project directly into **Google Antigravity**, you can set up a workspace folder on your computer with the pre-configured project files.

Below is the file structure to create, followed by the complete code and documentation files ready to copy and paste into your project directory.

Folder Setup

Create a root folder on your machine and set up the following directory layout:

```
agendaweb-app/
├── .antigravity/
│   └── rules.md                <-- System instructions & constraints for the Antigravity agent
├── docs/
│   ├── ARCHITECTURE.md         <-- System, data, and protocol specs
│   └── BRAND_GUIDELINES.md     <-- Colors, typography, icon geometry, and styling
├── public_html/
│   ├── index.html              <-- Client PWA interface
│   ├── feed.php                <-- Dual RSS 2.0 / ev: generator
```

```
│   ├── calendar.php              <-- Dynamic RFC 5545 iCal generator
│   ├── api/
│   │   └── extract.php           <-- "Read Later" OpenGraph link parser
│   ├── cron/
│   │   └── sync_sources.php      <-- Scraper & Australian public RSS ingester
│   └── assets/
│       └── logo.svg              <-- Production vector mark
└── schema.sql                    <-- MariaDB schema & Australian seed feeds
```

File 1: `.antigravity/rules.md`

Place this file inside `agendaweb-app/.antigravity/rules.md` to guide the Antigravity agent's boundaries and coding decisions.

```
# Antigravity Agent Rules: AgendaWeb

## Mission
Develop, test, and maintain AgendaWeb (`agendaweb.app`), an open-web, decentralized cultural calendar and editorial magazine reader for Australia.

## Core Directives
1. Zero User Authentication: Never introduce login barriers, passwords, or tracking databases for consumers. All favorites, feeds, and "Read Later" items must reside in the browser's IndexedDB.
2. Architecture Boundaries:
   - Backend: Stateless PHP 8.x + MariaDB running on Hostinger. Avoid node/npm dependencies on the server.
   - Frontend: Local-first Progressive Web App (HTML5, Vanilla JS or lightweight reactive framework, CSS variables).
   - Scrapers: Ingest events using native `schema.org/Event` (JSON-LD) and standard public Australian RSS feeds (ABC, SBS, The Conversation).
3. Calendar Standard: Support both single-event download (`calendar.php?id=X`) and continuous live subscriptions (`webcal://`).
4. Brand Integrity: Use only verified palette tokens:
   - Amber Rust: `#C85A32`
   - Sage Green: `#4A5D4E`
   - Nightfall: `#1B3B2B`
   - Chalk: `#F7F5F0`
   - Oat: `#E6E2D8`
```

File 2: `docs/ARCHITECTURE.md`

Place this file inside `agendaweb-app/docs/ARCHITECTURE.md`.

```
# AgendaWeb Architecture Specification

## Slogan
YOUR CULTURAL WEB FEED

## System Topology

[ Australian Media & Venue Web ] │ ├── Public RSS (ABC, SBS, The Conversation) └── Venue Webpages (Schema.org/Event JSON-LD) │ ▼ [ cron/sync_sources.php ] (Runs every 30m via Hostinger Cron) │ ▼ [ MariaDB Database ] ├── sources (Feed & scraper registry) ├── items (News, Politics, Culture articles) └── events (Gigs, exhibitions, screenings) │ ┌──────────┴──────────┐ ▼ ▼ [ feed.php ] [ calendar.php ] (RSS 2.0 + ev:startdate) (RFC 5545 .ics / webcal://) │ │ └──────────┬──────────┘ ▼ [ Client PWA Reader ] ├── Tab 1: ⚡ Events (Sorted by start date + category filters) ├── Tab 2: 📰 News (ABC/SBS public feeds) ├── Tab 3: 🏛 Politics (Public policy analysis) ├── Tab 4: 🎨 Culture (Essays, reviews, features) ├── Tab 5: 🔖 Reads ("Read Later" offline queue) └── Local Storage: IndexedDB (Zero server accounts)


## Protocol Specifications

### 1. Dual-Mode RSS 2.0 Feed (`feed.php`)
- Output: `application/rss+xml; charset=utf-8`
- Namespaces:
  - `xmlns:ev="http://purl.org/rss/1.0/modules/event/"`
  - `xmlns:atom="http://www.w3.org/2005/Atom"`
- Event Tags: `<ev:startdate>`, `<ev:enddate>`, `<ev:location>`
- Enclosure: `<enclosure url=".../calendar.php?id=X" type="text/calendar" />`

### 2. Live Calendar Protocol (`calendar.php`)
- Content-Type: `text/calendar; charset=utf-8`
- Standard: RFC 5545 (iCalendar)
- Protocol: Supports HTTP direct download and `webcal://` persistent calendar subscriptions.

### 3. Read Later Link Extractor (`api/extract.php`)
- Accepts: `GET /api/extract.php?url={external_url}`
- Response: JSON payload containing OpenGraph tags (`og:title`, `og:description`, `og:image`, `domain`).
- Storage: Saved exclusively on client `IndexedDB` (`read_later` store).
```

File 3: `docs/BRAND_GUIDELINES.md`

Place this file inside `agendaweb-app/docs/BRAND_GUIDELINES.md`.

```
# AgendaWeb Brand Guidelines

## Color Palette Tokens

| Name | Hex | RGB | HSL | Role |
| :--- | :--- | :--- | :--- | :--- |
| **Amber Rust** | `#C85A32` | `200, 90, 50` | `16°, 60%, 49%` | Brand bookmark ribbon, primary CTA buttons |
| **Sage Green** | `#4A5D4E` | `74, 93, 78` | `133°, 11%, 33%` | RSS wave arcs, active tags, live indicators |
| **Nightfall** | `#1B3B2B` | `27, 59, 43` | `150°, 37%, 17%` | Calendar binder rings, grid blocks, primary headings |
| **Chalk** | `#F7F5F0` | `247, 245, 240` | `43°, 27%, 95%` | Main canvas, broadsheet paper background |
| **Oat** | `#E6E2D8` | `230, 226, 216` | `43°, 18%, 87%` | Borders, subtle panels, card dividers |

## Typography
- **Headings & Brand Title:** `Newsreader` or `Playfair Display` (Serif, weights 700/800)
- **UI Elements, Schedules & Body:** `Inter` or `Plus Jakarta Sans` (Sans-serif, weights 400/500/600/800)

## Logo Geometry & Iconography
The unified mark consists of:
1. **Bookmark Ribbon:** Solid `#C85A32` vertical badge with a square (`90°`) top-left corner, a rounded top-right shoulder, and a clean 'V' cut base (no bottom stripe line).
2. **RSS Wave Arcs:** Two rounded concentric arcs in `#4A5D4E` radiating from the top-right rounded corner.
3. **Calendar Face:** Centered `#F7F5F0` plate with two `#1B3B2B` binder rings at top.
4. **Grid Pattern:** 7 asymmetrical blocks in `#1B3B2B` arranged across 3 columns (left: 2, center: 3, right: 2).
```

File 4: `public_html/assets/logo.svg`

Place this file inside `agendaweb-app/public_html/assets/logo.svg`.

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 520 180" width="100%" height="100%" fill="none">
  <!-- RSS Waves (Top Right) -->
  <path d="M 125 36 A 48 48 0 0 1 173 84" stroke="#4A5D4E" stroke-width="10" stroke-linecap="round" />
  <path d="M 125 18 A 66 66 0 0 1 191 84" stroke="#4A5D4E" stroke-width="10" stroke-linecap="round" />

  <!-- Bookmark Ribbon (Square top-left, round top-right, clean V bottom) -->
  <path d="M 45 42 L 122 42 C 137 42 145 50 145 65 L 145 165 L 95 130 L 45 165 Z" fill="#C85A32" />

  <!-- Calendar Container -->
  <rect x="58" y="70" width="74" height="54" rx="4" fill="#F7F5F0" />

  <!-- Binder Rings -->
  <rect x="70" y="63" width="7" height="15" rx="3.5" fill="#1B3B2B" />
  <rect x="109" y="63" width="7" height="15" rx="3.5" fill="#1B3B2B" />

  <!-- Calendar Grid (7 Asymmetrical Blocks) -->
  <g fill="#1B3B2B">
    <!-- Col 1 -->
    <rect x="66" y="80" width="12" height="10" rx="1.5" />
    <rect x="66" y="94" width="12" height="10" rx="1.5" />
    <!-- Col 2 -->
    <rect x="82" y="80" width="12" height="10" rx="1.5" />
    <rect x="82" y="94" width="12" height="10" rx="1.5" />
    <rect x="82" y="108" width="12" height="10" rx="1.5" />
    <!-- Col 3 -->
    <rect x="98" y="80" width="12" height="10" rx="1.5" />
    <rect x="98" y="94" width="12" height="10" rx="1.5" />
  </g>

  <!-- Typography -->
  <text x="175" y="98" font-family="&#39;Inter&#39;, sans-serif" font-size="42" font-weight="800" fill="#1B3B2B" letter-spacing="1.2">AGENDAWEB</text>
  <text x="178" y="122" font-family="&#39;Inter&#39;, sans-serif" font-size="13" font-weight="600" fill="#1B3B2B" letter-spacing="2.2">YOUR CULTURAL WEB FEED</text>
</svg>
```

File 5: `schema.sql`

Place this file inside `agendaweb-app/schema.sql`.

```
-- AgendaWeb Database Schema for MariaDB (Hostinger)

CREATE TABLE IF NOT EXISTS `sources` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
```

```
  `slug` VARCHAR(60) UNIQUE NOT NULL,
  `name` VARCHAR(120) NOT NULL,
  `type` ENUM('rss', 'scrape_event') NOT NULL DEFAULT 'rss',
  `url` VARCHAR(255) NOT NULL,
  `category_tab` ENUM('events', 'news', 'politics', 'culture') NOT NULL,
  `is_active` TINYINT(1) DEFAULT 1,
  `last_polled_at` DATETIME NULL,
  INDEX (`category_tab`),
  INDEX (`is_active`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS `events` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `venue_slug` VARCHAR(60) NOT NULL,
  `title` VARCHAR(255) NOT NULL,
  `description` TEXT,
  `category` VARCHAR(60) DEFAULT 'Culture',
  `start_time` DATETIME NOT NULL,
  `end_time` DATETIME DEFAULT NULL,
  `venue_name` VARCHAR(150) NOT NULL,
  `address` VARCHAR(255) DEFAULT NULL,
  `ticket_url` VARCHAR(255) DEFAULT NULL,
  `origin` ENUM('scraped', 'manual') DEFAULT 'scraped',
  `is_verified` TINYINT(1) DEFAULT 0,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY `idx_venue_event_time` (`venue_slug`, `title`(100), `start_time`),
  INDEX `idx_start_time` (`start_time`),
  INDEX `idx_venue_slug` (`venue_slug`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS `items` (
  `id` INT AUTO_INCREMENT PRIMARY KEY,
  `source_id` INT NOT NULL,
  `category_tab` ENUM('news', 'politics', 'culture') NOT NULL,
  `title` VARCHAR(255) NOT NULL,
  `link` VARCHAR(255) UNIQUE NOT NULL,
  `summary` TEXT,
  `published_at` DATETIME NOT NULL,
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (`source_id`) REFERENCES `sources`(`id`) ON DELETE CASCADE,
  INDEX `idx_published_at` (`published_at`),
  INDEX `idx_tab` (`category_tab`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Initial Seed: Public Australian RSS & Event Targets
INSERT INTO `sources` (`slug`, `name`, `type`, `url`, `category_tab`) VALUES
('abc-just-in', 'ABC News', 'rss', 'https://www.abc.net.au/news/feed/51120/rss.xml', 'news'),
('sbs-top-stories', 'SBS News', 'rss', 'https://www.sbs.com.au/news/feed', 'news'),
('abc-politics', 'ABC Politics', 'rss', 'https://www.abc.net.au/news/feed/45910/rss.xml', 'politics'),
('the-conversation-au', 'The Conversation (AU)', 'rss', 'https://theconversation.com/au/politics/articles.atom', 'politics'),
('guardian-au-culture', 'The Guardian (AU Culture)', 'rss', 'https://www.theguardian.com/au/culture/rss', 'culture'),
('the-gasometer-hotel', 'The Gasometer Hotel', 'scrape_event', 'https://www.thegasometerhotel.com.au/gigs', 'events')
ON DUPLICATE KEY UPDATE `name`=VALUES(`name`);
```

File 6: `public_html/feed.php`

Place this file inside `agendaweb-app/public_html/feed.php`.

```
<?php
declare(strict_types=1);

header('Content-Type: application/rss+xml; charset=utf-8');
header('Cache-Control: public, max-age=300');

$host = 'localhost';
$db   = 'u123456789_agenda';
$user = 'u123456789_admin';
$pass = 'YourSecretPasswordHere';

try {
    $pdo = new PDO("mysql:host={$host};dbname={$db};charset=utf8mb4", $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
    ]);
} catch (PDOException $e) {
    http_response_code(500);
    exit('<error>Database Connection Failed</error>');
}

$venue = filter_input(INPUT_GET, 'venue', FILTER_DEFAULT) ?? 'all';
$category = filter_input(INPUT_GET, 'category', FILTER_DEFAULT);

$sql = "SELECT * FROM events WHERE start_time >= NOW()";
$params = [];

if ($venue !== 'all') {
    $sql .= " AND venue_slug = :venue";
    $params['venue'] = $venue;
}
if (!empty($category) && $category !== 'All') {
    $sql .= " AND category = :cat";
    $params['cat'] = $category;
}

$sql .= " ORDER BY start_time ASC LIMIT 60";
$stmt = $pdo->prepare($sql);
$stmt->execute($params);
$events = $stmt->fetchAll();

echo '<?xml version="1.0" encoding="UTF-8"?>' . PHP_EOL;
?>
<rss version="2.0"
     xmlns:ev="http://purl.org/rss/1.0/modules/event/"
     xmlns:atom="http://www.w3.org/2005/Atom">
  <channel>
    <title>AgendaWeb — <?= htmlspecialchars(ucfirst($venue)) ?></title>
    <link>https://agendaweb.app</link>
    <description>YOUR CULTURAL WEB FEED</description>
    <language>en-AU</language>
    <atom:link href="https://agendaweb.app/feed.php" rel="self" type="application/rss+xml" />

    <?php foreach ($events as $row): ?>
    <item>
      <title><?= htmlspecialchars($row['title'], ENT_XML1, 'UTF-8') ?></title>
      <link><?= htmlspecialchars($row['ticket_url'] ?: 'https://agendaweb.app', ENT_XML1, 'UTF-8') ?></link>
      <pubDate><?= date(DATE_RSS, strtotime($row['created_at'])) ?></pubDate>
      <category><?= htmlspecialchars($row['category'], ENT_XML1, 'UTF-8') ?></category>
      <description><![CDATA[<?= $row['description'] ?>]]></description>

      <ev:startdate><?= date('c', strtotime($row['start_time'])) ?></ev:startdate>
      <?php if (!empty($row['end_time'])): ?>
      <ev:enddate><?= date('c', strtotime($row['end_time'])) ?></ev:enddate>
      <?php endif; ?>
      <ev:location><?= htmlspecialchars($row['venue_name'] . ' · ' . $row['address'], ENT_XML1, 'UTF-8') ?></ev:location>
      <enclosure url="https://agendaweb.app/calendar.php?id=<?= $row['id'] ?>" type="text/calendar" />
    </item>
    <?php endforeach; ?>
  </channel>
</rss>
```

File 7: `public_html/calendar.php`

Place this file inside `agendaweb-app/public_html/calendar.php`.

```
<?php
declare(strict_types=1);

$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);
$venue = filter_input(INPUT_GET, 'venue', FILTER_DEFAULT);

$pdo = new PDO('mysql:host=localhost;dbname=u123456789_agenda;charset=utf8mb4', 'u123456789_admin', 'YourSecretPasswordHere');

if ($id) {
```

```
    $stmt = $pdo->prepare("SELECT * FROM events WHERE id = ?");
    $stmt->execute([$id]);
    $events = $stmt->fetchAll(PDO::FETCH_ASSOC);
    $filename = "agendaweb-event-{$id}.ics";
} elseif ($venue) {
    $stmt = $pdo->prepare("SELECT * FROM events WHERE (venue_slug = :v OR :v = 'all') AND start_time >= NOW() ORDER BY start_time ASC LIMIT 100");
    $stmt->execute(['v' => $venue]);
    $events = $stmt->fetchAll(PDO::FETCH_ASSOC);
    $filename = "agendaweb-{$venue}.ics";
} else {
    http_response_code(400);
    exit('Invalid Event or Venue Parameter');
}

header('Content-Type: text/calendar; charset=utf-8');
header("Content-Disposition: attachment; filename=\"{$filename}\"");

echo "BEGIN:VCALENDAR\r\n";
echo "VERSION:2.0\r\n";
echo "PRODID:-//AgendaWeb//agendaweb.app//EN\r\n";
echo "CALSCALE:GREGORIAN\r\n";

foreach ($events as $e) {
    $dtstamp = gmdate('Ymd\THis\Z');
    $start   = gmdate('Ymd\THis\Z', strtotime($e['start_time']));
    $end     = !empty($e['end_time']) ? gmdate('Ymd\THis\Z', strtotime($e['end_time'])) : gmdate('Ymd\THis\Z', strtotime($e['start_time'] . ' +2 hours'));

    echo "BEGIN:VEVENT\r\n";
    echo "UID:agendaweb-{$e['id']}@agendaweb.app\r\n";
    echo "DTSTAMP:{$dtstamp}\r\n";
    echo "DTSTART:{$start}\r\n";
    echo "DTEND:{$end}\r\n";
    echo "SUMMARY:" . addcslashes($e['title'], ",\\;") . "\r\n";
    echo "LOCATION:" . addcslashes($e['venue_name'] . ', ' . $e['address'], ",\\;") . "\r\n";
    echo "DESCRIPTION:" . addcslashes($e['description'] . "\n\nTickets: " . ($e['ticket_url'] ?: ''), ",\\;") . "\r\n";
    if (!empty($e['ticket_url'])) {
        echo "URL:{$e['ticket_url']}\r\n";
    }
    echo "END:VEVENT\r\n";
}

echo "END:VCALENDAR\r\n";
```

File 8: `public_html/api/extract.php`

Place this file inside `agendaweb-app/public_html/api/extract.php`.

```
<?php
declare(strict_types=1);

header('Content-Type: application/json; charset=utf-8');
header('Access-Control-Allow-Origin: *');

$targetUrl = filter_input(INPUT_GET, 'url', FILTER_VALIDATE_URL);
if (!$targetUrl) {
    http_response_code(400);
    echo json_encode(['error' => 'Valid URL required']);
    exit;
}

$ctx = stream_context_create([
    'http' => [
        'timeout' => 6,
        'user_agent' => 'AgendaWeb-Bot/1.0 (+https://agendaweb.app)'
    ]
]);

$html = @file_get_contents($targetUrl, false, $ctx, 0, 65536);
if ($html === false) {
    http_response_code(422);
    echo json_encode(['error' => 'Unable to read target page']);
    exit;
}

libxml_use_internal_errors(true);
$dom = new DOMDocument();
@$dom->loadHTML($html);
libxml_clear_errors();

$xpath = new DOMXPath($dom);
$queryMeta = function(string $prop) use ($xpath): string {
    $node = $xpath->query("//meta[@property='{$prop}']/@content | //meta[@name='{$prop}']/@content");
    return ($node && $node->length > 0) ? trim($node->item(0)->nodeValue) : '';
};

$title = $queryMeta('og:title') ?: $queryMeta('twitter:title');
if (!$title) {
    $tNodes = $xpath->query('//title');
    $title = ($tNodes && $tNodes->length > 0) ? trim($tNodes->item(0)->nodeValue) : 'Untitled Link';
}

$description = $queryMeta('og:description') ?: $queryMeta('description');
$image       = $queryMeta('og:image');
$siteName    = $queryMeta('og:site_name') ?: parse_url($targetUrl, PHP_URL_HOST);

echo json_encode([
    'url'         => $targetUrl,
    'title'       => html_entity_decode($title, ENT_QUOTES, 'UTF-8'),
    'description' => html_entity_decode($description, ENT_QUOTES, 'UTF-8'),
    'image'       => $image,
    'domain'      => $siteName,
    'saved_at'    => gmdate('Y-m-d\TH:i:s\Z')
], JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE);
```

File 9: `public_html/cron/sync_sources.php`

Place this file inside `agendaweb-app/public_html/cron/sync_sources.php`.

```
<?php
declare(strict_types=1);
set_time_limit(300);

$pdo = new PDO(
    'mysql:host=localhost;dbname=u123456789_agenda;charset=utf8mb4',
    'u123456789_admin',
    'YourSecretPasswordHere',
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);

$sources = $pdo->query("SELECT * FROM sources WHERE is_active = 1")->fetchAll(PDO::FETCH_ASSOC);

foreach ($sources as $source) {
    if ($source['type'] === 'rss') {
        syncRss($pdo, $source);
    } elseif ($source['type'] === 'scrape_event') {
        syncSchemaOrgEvents($pdo, $source);
    }
    $pdo->prepare("UPDATE sources SET last_polled_at = NOW() WHERE id = ?")->execute([$source['id']]);
}

function syncRss(PDO $pdo, array $source): void {
    $ctx = stream_context_create(['http' => ['timeout' => 10, 'user_agent' => 'AgendaWeb-Sync/1.0']]);
    $xmlData = @file_get_contents($source['url'], false, $ctx);
    if (!$xmlData) return;

    $xml = @simplexml_load_string($xmlData);
    if (!$xml) return;

    $stmt = $pdo->prepare("
        INSERT INTO items (source_id, category_tab, title, link, summary, published_at)
        VALUES (:sid, :tab, :title, :link, :summary, :pub_date)
        ON DUPLICATE KEY UPDATE summary = VALUES(summary)
    ");
```

```
    $items = isset($xml->channel->item) ? $xml->channel->item : ($xml->entry ?? []);
    $i = 0;
    foreach ($items as $item) {
        if ($i++ > 20) break;
        $title = (string)$item->title;
        $link  = (string)($item->link['href'] ?? $item->link);
        $desc  = strip_tags((string)($item->description ?? $item->summary ?? ''));
        $date  = (string)($item->pubDate ?? $item->updated ?? date('r'));

        $stmt->execute([
            'sid'      => $source['id'],
            'tab'      => $source['category_tab'],
            'title'    => $title,
            'link'     => $link,
            'summary'  => mb_substr($desc, 0, 300) . '...',
            'pub_date' => date('Y-m-d H:i:s', strtotime($date))
        ]);
    }
}

function syncSchemaOrgEvents(PDO $pdo, array $source): void {
    $ctx = stream_context_create(['http' => ['timeout' => 12, 'user_agent' => 'AgendaWeb-Bot/1.0']]);
    $html = @file_get_contents($source['url'], false, $ctx);
    if (!$html) return;

    libxml_use_internal_errors(true);
    $dom = new DOMDocument();
    @$dom->loadHTML($html);
    libxml_clear_errors();

    $xpath = new DOMXPath($dom);
    $scripts = $xpath->query('//script[@type="application/ld+json"]');

    $stmt = $pdo->prepare("
        INSERT INTO events (venue_slug, title, description, category, start_time, end_time, venue_name, address, ticket_url)
        VALUES (:slug, :title, :description, :cat, :start, :end, :vname, :addr, :turl)
        ON DUPLICATE KEY UPDATE description = VALUES(description), ticket_url = VALUES(ticket_url)
    ");

    foreach ($scripts as $s) {
        $data = json_decode(trim($s->nodeValue), true);
        if (!is_array($data)) continue;

        $nodes = isset($data['@graph']) ? $data['@graph'] : ($data[0] ? $data : [$data]);
        foreach ($nodes as $node) {
            $type = $node['@type'] ?? '';
            if (is_string($type) && (str_ends_with($type, 'Event') || $type === 'Event')) {
                if (empty($node['name']) || empty($node['startDate'])) continue;

                $start = date('Y-m-d H:i:s', strtotime($node['startDate']));
                if (strtotime($start) < time()) continue;

                $stmt->execute([
                    'slug'        => $source['slug'],
                    'title'       => trim($node['name']),
                    'description' => strip_tags($node['description'] ?? ''),
                    'cat'         => str_replace('Event', '', (string)$type) ?: 'Culture',
                    'start'       => $start,
                    'end'         => !empty($node['endDate']) ? date('Y-m-d H:i:s', strtotime($node['endDate'])) : null,
                    'vname'       => $node['location']['name'] ?? $source['name'],
                    'addr'        => is_array($node['location']['address'] ?? null) ? ($node['location']['address']['streetAddress'] ?? '') : ($node['location']['address'] ?? ''),
                    'turl'        => $node['offers']['url'] ?? $node['url'] ?? $source['url']
                ]);
            }
        }
    }
}
```

File 10: `public_html/index.html`

Place this file inside `agendaweb-app/public_html/index.html` to preview the PWA interface.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>AgendaWeb — Your Cultural Web Feed</title>
  <link rel="icon" type="image/svg+xml" href="assets/logo.svg">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=Playfair+Display:wght@700&display=swap" rel="stylesheet">

  <style>
    :root {
      --color-amber-rust: #C85A32;
      --color-sage-green: #4A5D4E;
      --color-nightfall:  #1B3B2B;
      --color-chalk:      #F7F5F0;
      --color-oat:        #E6E2D8;
    }
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      background-color: var(--color-chalk);
      color: var(--color-nightfall);
      font-family: 'Inter', sans-serif;
      line-height: 1.4;
      padding-bottom: 80px;
    }
    header {
      background: #FFFFFF;
      border-bottom: 1px solid var(--color-oat);
      padding: 14px 18px;
      position: sticky;
      top: 0;
      z-index: 100;
      display: flex;
      align-items: center;
      justify-content: space-between;
    }
    .brand {
      display: flex;
      align-items: center;
      gap: 12px;
      text-decoration: none;
    }
    .brand img { height: 42px; }
    .tabs {
      display: flex;
      overflow-x: auto;
      gap: 8px;
      padding: 12px 18px;
      background: var(--color-chalk);
      border-bottom: 1px solid var(--color-oat);
    }
    .tab-btn {
      background: #FFFFFF;
      border: 1px solid var(--color-oat);
      padding: 8px 14px;
      border-radius: 20px;
      font-size: 0.85rem;
      font-weight: 600;
      color: var(--color-nightfall);
      white-space: nowrap;
      cursor: pointer;
    }
    .tab-btn.active {
      background: var(--color-sage-green);
      color: #FFFFFF;
      border-color: var(--color-sage-green);
    }
    .sub-filter {
      display: flex;
```

```
      overflow-x: auto;
      gap: 6px;
      padding: 8px 18px;
      font-size: 0.8rem;
    }
    .filter-pill {
      background: transparent;
      border: none;
      padding: 4px 8px;
      color: var(--color-nightfall);
      opacity: 0.7;
      cursor: pointer;
    }
    .filter-pill.active {
      opacity: 1;
      font-weight: 700;
      border-bottom: 2px solid var(--color-amber-rust);
    }
    .feed-container {
      max-width: 640px;
      margin: 16px auto;
      padding: 0 16px;
    }
    .card {
      background: #FFFFFF;
      border: 1px solid var(--color-oat);
      border-radius: 12px;
      padding: 16px;
      margin-bottom: 14px;
      box-shadow: 0 2px 4px rgba(27, 59, 43, 0.03);
    }
    .card-meta {
      font-size: 0.8rem;
      font-weight: 700;
      color: var(--color-amber-rust);
      text-transform: uppercase;
      letter-spacing: 0.5px;
    }
    .card-title {
      font-size: 1.15rem;
      font-weight: 700;
      margin: 4px 0 6px 0;
      color: var(--color-nightfall);
    }
    .card-location {
      font-size: 0.85rem;
      color: #5C675E;
      margin-bottom: 12px;
    }
    .card-actions {
      display: flex;
      gap: 8px;
    }
    .btn {
      padding: 8px 12px;
      font-size: 0.8rem;
      font-weight: 600;
      border-radius: 6px;
      text-decoration: none;
      cursor: pointer;
      display: inline-flex;
      align-items: center;
      gap: 4px;
    }
    .btn-primary { background: var(--color-nightfall); color: #FFFFFF; border: none; }
    .btn-secondary { background: var(--color-amber-rust); color: #FFFFFF; border: none; }
    .btn-outline { background: transparent; border: 1px solid var(--color-oat); color: var(--color-nightfall); }
  </style>
</head>
<body>

  <header>
    <a href="#" class="brand">
      <img src="assets/logo.svg" alt="AgendaWeb">
    </a>
    <div style="font-size: 0.8rem; color: var(--color-sage-green); font-weight: 600;">✦ Open Feed</div>
  </header>

  <nav class="tabs">
    <button class="tab-btn active" onclick="setTab('events')">⚡ Events</button>
    <button class="tab-btn" onclick="setTab('news')">📰 News</button>
    <button class="tab-btn" onclick="setTab('politics')">🏛 Politics</button>
    <button class="tab-btn" onclick="setTab('culture')">🎨 Culture</button>
    <button class="tab-btn" onclick="setTab('reads')">🔖 Reads</button>
  </nav>

  <div class="sub-filter" id="sub-filter">
    <button class="filter-pill active">All</button>
    <button class="filter-pill">Live Music</button>
    <button class="filter-pill">Art & Fair</button>
    <button class="filter-pill">Community</button>
  </div>

  <main class="feed-container" id="feed">
    <!-- Sample Event Card -->
    <article class="card">
      <div class="card-meta">Fri, 24 Oct · Doors 8:00 PM</div>
      <h2 class="card-title">The Echoes — Album Launch</h2>
      <div class="card-location">📍 The Gasometer Hotel, Collingwood</div>
      <div class="card-actions">
        <button class="btn btn-primary" onclick="window.location.href='calendar.php?id=1'">📅 Add to Calendar</button>
        <button class="btn btn-secondary">★ Save</button>
        <a href="#" class="btn btn-outline" target="_blank">Tickets ($22)</a>
      </div>
    </article>
  </main>

  <script>
    function setTab(tabName) {
      document.querySelectorAll('.tab-btn').forEach(btn => btn.classList.remove('active'));
      event.target.classList.add('active');
      document.getElementById('sub-filter').style.display = (tabName === 'events') ? 'flex' : 'none';
    }
  </script>
</body>
</html>
```

Step 3: Launching in Google Antigravity

1. In the **Google Antigravity App** or **CLI**, open the workspace:

   ```
   antigravity .
   ```

   *(or select **Open Project** in the Antigravity desktop interface and pick the `agendaweb-app` folder).*

2. Antigravity will detect `.antigravity/rules.md` and parse the project boundary.

3. Start your session with this kick-off prompt:

   > *"I have seeded the architecture, rules, schemas, and endpoints into this workspace. Read `.antigravity/rules.md` and `docs/ARCHITECTURE.md`. Generate an implementation plan to run local tests on `feed.php` and `cron/sync_sources.php`, verify that the RSS event output conforms to `<ev:startdate>`, and wire up the PWA's IndexedDB storage for the 'Reads' tab."*
